1. Problems with JavaScript and how TypeScript solves them

1. No strict type checking by default

JavaScript allows a variable to hold different types of values, which can lead to unexpected behavior.

JavaScript example:

```
let age = 21;

age = "twenty-one"; // No error!

console.log(age);
```

JavaScript allows us to change age from a number to a string, even if our application expects a number.

TypeScript solution:

```
let age: number = 21;

age = "twenty-one"; // TypeScript error
```

TypeScript detects the incorrect assignment before the code runs.

Benefit: Helps prevent type-related bugs.

2. Errors are often discovered at runtime

In JavaScript, some errors only appear when a particular line of code executes.

JavaScript example:

```
function add(a, b) {
  return a + b;
}

console.log(add(10, 20));  // 30
console.log(add(10, "20")); // "1020"
```

You might expect 10 + "20" to produce 30, but JavaScript performs string concatenation and returns "1020".

TypeScript solution:

```
function add(a: number, b: number): number {
  return a + b;
}

console.log(add(10, 20));   // 30
console.log(add(10, "20")); // TypeScript error
```

Benefit: Catches many mistakes during development rather than waiting for users to encounter them.

Note: TypeScript catches many errors before runtime, but it cannot eliminate every possible runtime error.

3. No automatic checking of object properties

JavaScript does not warn you when you access a property that doesn't exist on an object.

JavaScript example:

```
const user = {
  name: "Karan",
  age: 21
};

console.log(user.email);
// undefined
```

You might accidentally use email instead of another property, and JavaScript won't flag it.

TypeScript solution:

```
type User = {
  name: string;
  age: number;
};

const user: User = {
  name: "Karan",
  age: 21
};

console.log(user.email);
// TypeScript error: email doesn't exist
```

Benefit: Makes working with objects, API responses, and application data safer.

4. Function arguments are not checked automatically

JavaScript functions don't require you to declare the types of their arguments.

JavaScript example:

```
function calculatePrice(price, quantity) {
  return price * quantity;
}

console.log(calculatePrice("100", "2"));
```

JavaScript may convert these strings to numbers automatically, which can hide incorrect input types.

TypeScript solution:

```
function calculatePrice(
  price: number,
  quantity: number
): number {
  return price * quantity;
}

calculatePrice(100, 2);       // Valid
calculatePrice("100", "2");   // TypeScript error
```

> **Benefit:** Makes function expectations clear and prevents many invalid calls.

5. Poor support for understanding large codebases

In a large JavaScript project, you may not know what type of data a function expects or returns just by looking at its name.

JavaScript example:

```
function getUser() {
  return {
    name: "Karan",
    age: 21
  };
}

const user = getUser();
```

The function's return structure isn't explicitly documented through types.

TypeScript solution:

```
type User = {
  name: string;
  age: number;
};

function getUser(): User {
  return {
    name: "Karan",
    age: 21
  };
}

const user = getUser();
```

Now, developers can immediately see the expected structure of the returned data.

> **Benefit:** Improves readability, maintainability, and collaboration in large teams.

6. Refactoring can be risky

Refactoring means changing existing code without changing its intended behavior.

Suppose you rename a property from name to fullName.

JavaScript example:

```
const user = {
  name: "Karan"
};

console.log(user.name);

// Later, you change the object:
const updatedUser = {
  fullName: "Karan"
};

console.log(updatedUser.name); // undefined
```

JavaScript doesn't automatically identify every place where your old property name is still being used.

TypeScript solution:

```
type User = {
  fullName: string;
};

const user: User = {
  fullName: "Karan"
};

console.log(user.name); // TypeScript error
```

Your editor can also use TypeScript's language services to help rename symbols and identify affected references.

> **Benefit:** Makes changing large applications safer and easier.

7. API data can be used incorrectly

When fetching data from an API, JavaScript doesn't automatically verify that the returned data matches the structure you expect.

JavaScript example:

```
const movie = {
  title: "Inception",
  rating: 8.8
};

console.log(movie.rating.toUpperCase());
// Runtime error: toUpperCase is not a function
```

The rating is a number, not a string.

TypeScript solution:

```
type Movie = {
  title: string;
  rating: number;
};

const movie: Movie = {
  title: "Inception",
  rating: 8.8
};

console.log(movie.rating.toUpperCase());
// TypeScript error
```

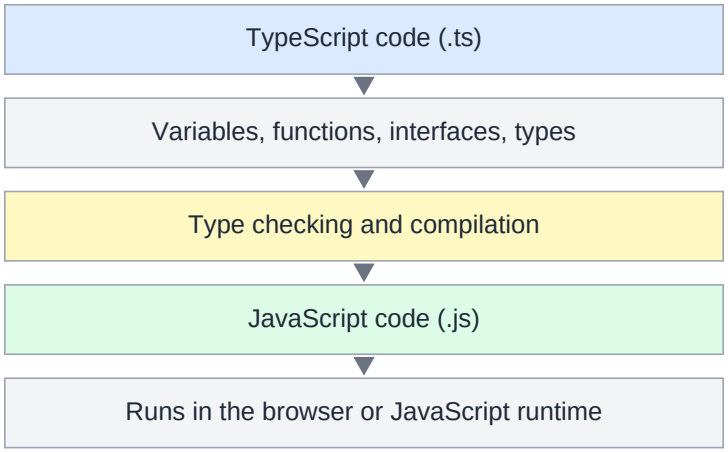
Important: TypeScript types alone do not validate external API responses at runtime. For untrusted data, you should also use runtime validation when necessary.

Benefit: Helps you work with API data more reliably.

8. JavaScript has no built-in compile-time type system

JavaScript typically checks types through runtime behavior rather than a static type system.

TypeScript adds a development-time checking layer.



For example:

```
let score: number = 100;
console.log(score);
```

TypeScript checks the type and removes type annotations when compiling to JavaScript. The browser does not need to understand the : number annotation.

Benefit: You get static checking while retaining the JavaScript ecosystem.

2. Quick comparison: JavaScript vs TypeScript

Feature	JavaScript	TypeScript
Type checking	Primarily at runtime	Static checking during development
Type annotations	Not supported in standard JS syntax	Supported
Detecting invalid properties	Often at runtime	Often caught before runtime
Function argument types	Not declared in standard syntax	Can be explicitly declared
IDE autocomplete	Available	Often more precise with types
Refactoring	Can be riskier in large projects	Type-aware tools can help
Learning curve	Easier to start	Requires learning types
Browser execution	Runs directly	Usually compiled/transformed to JavaScript

3. Does TypeScript replace JavaScript?

No. TypeScript is built on top of JavaScript.

- JavaScript provides the core language and runtime behavior.
- TypeScript adds a static type system and development tools.
- TypeScript is converted to JavaScript before execution in typical browser applications.
- Valid JavaScript code is generally valid TypeScript code, though TypeScript's checks and project settings may report issues.

4. When should you use TypeScript?

Use TypeScript especially when:

- Building large React applications.
- Working with APIs and complex data structures.
- Collaborating with other developers.
- Maintaining code over a long period.
- Refactoring or changing shared components and functions.

JavaScript is still perfectly useful for small scripts, prototypes, and projects where its simplicity is an advantage.

> **Interview-ready answer:**
>
> *"JavaScript is dynamically typed, so many type-related errors are discovered only at runtime. It also doesn't provide built-in static checking for object properties or function arguments. TypeScript addresses these limitations by adding static typing, interfaces, and other type-system features. This improves code reliability, IDE support, and maintainability, especially in large applications. However, TypeScript doesn't replace JavaScript; it compiles to JavaScript and still requires runtime testing and validation."*